

```
div2_lab_HIGH_PERFORMANCE — -zsh — 144x39

stic/div2_lab_HIGH_PERFORMANCE
(base) dimasaltynbek@MacBook-Pro-Dimas div2_lab_HIGH_PERFORMANCE % python3 fork_join_lab1.py determinism

===== RUN 1 =====
--- Forking a team of 4 threads ---
[Master] Logical Rank: 0 of 4 | Native OS TID: 3815349
[Worker] Logical Rank: 3 of 4 | Native OS TID: 3815351
[Worker] Logical Rank: 1 of 4 | Native OS TID: 3815349
[Worker] Logical Rank: 2 of 4 | Native OS TID: 3815350
--- Joined thread team. Execution returned to serial master ---

===== RUN 2 =====
--- Forking a team of 4 threads ---
[Master] Logical Rank: 0 of 4 | Native OS TID: 3815352
[Worker] Logical Rank: 3 of 4 | Native OS TID: 3815354
[Worker] Logical Rank: 1 of 4 | Native OS TID: 3815352
[Worker] Logical Rank: 2 of 4 | Native OS TID: 3815353
--- Joined thread team. Execution returned to serial master ---

===== RUN 3 =====
--- Forking a team of 4 threads ---
[Master] Logical Rank: 0 of 4 | Native OS TID: 3815355
[Worker] Logical Rank: 3 of 4 | Native OS TID: 3815357
[Worker] Logical Rank: 1 of 4 | Native OS TID: 3815355
[Worker] Logical Rank: 2 of 4 | Native OS TID: 3815356
--- Joined thread team. Execution returned to serial master ---

===== RUN 4 =====
--- Forking a team of 4 threads ---
[Master] Logical Rank: 0 of 4 | Native OS TID: 3815358
[Worker] Logical Rank: 3 of 4 | Native OS TID: 3815360
[Worker] Logical Rank: 1 of 4 | Native OS TID: 3815358
[Worker] Logical Rank: 2 of 4 | Native OS TID: 3815359
--- Joined thread team. Execution returned to serial master ---
```

```
div2_lab_HIGH_PERFORMANCE — -zsh — 144x39

===== RUN 5 =====
--- Forking a team of 4 threads ---
[Master] Logical Rank: 0 of 4 | Native OS TID: 3815361
[Worker] Logical Rank: 3 of 4 | Native OS TID: 3815363
[Worker] Logical Rank: 1 of 4 | Native OS TID: 3815361
[Worker] Logical Rank: 2 of 4 | Native OS TID: 3815362
--- Joined thread team. Execution returned to serial master ---

===== RUN 6 =====
--- Forking a team of 4 threads ---
[Master] Logical Rank: 0 of 4 | Native OS TID: 3815364
[Worker] Logical Rank: 3 of 4 | Native OS TID: 3815366
[Worker] Logical Rank: 1 of 4 | Native OS TID: 3815364
[Worker] Logical Rank: 2 of 4 | Native OS TID: 3815365
--- Joined thread team. Execution returned to serial master ---

===== RUN 7 =====
--- Forking a team of 4 threads ---
[Master] Logical Rank: 0 of 4 | Native OS TID: 3815367
[Worker] Logical Rank: 3 of 4 | Native OS TID: 3815369
[Worker] Logical Rank: 1 of 4 | Native OS TID: 3815367
[Worker] Logical Rank: 2 of 4 | Native OS TID: 3815368
--- Joined thread team. Execution returned to serial master ---

===== RUN 8 =====
--- Forking a team of 4 threads ---
[Master] Logical Rank: 0 of 4 | Native OS TID: 3815370
[Worker] Logical Rank: 3 of 4 | Native OS TID: 3815372
[Worker] Logical Rank: 1 of 4 | Native OS TID: 3815370
[Worker] Logical Rank: 2 of 4 | Native OS TID: 3815371
--- Joined thread team. Execution returned to serial master ---

===== RUN 9 =====
--- Forking a team of 4 threads ---

===== RUN 9 =====
--- Forking a team of 4 threads ---
[Master] Logical Rank: 0 of 4 | Native OS TID: 3815373
[Worker] Logical Rank: 3 of 4 | Native OS TID: 3815375
[Worker] Logical Rank: 1 of 4 | Native OS TID: 3815373
[Worker] Logical Rank: 2 of 4 | Native OS TID: 3815374
--- Joined thread team. Execution returned to serial master ---

===== RUN 10 =====
--- Forking a team of 4 threads ---
[Master] Logical Rank: 0 of 4 | Native OS TID: 3815376
[Worker] Logical Rank: 3 of 4 | Native OS TID: 3815378
[Worker] Logical Rank: 1 of 4 | Native OS TID: 3815376
[Worker] Logical Rank: 2 of 4 | Native OS TID: 3815377
--- Joined thread team. Execution returned to serial master ---

Saved raw stdout of all 10 runs to task1_1_determinism_log.txt
(base) dimasaltynbek@MacBook-Pro-Dimas div2_lab_HIGH_PERFORMANCE %
```

task 1.2

```
((base) dimasaltynbek@MacBook-Pro-Dimas div2_lab_HIGH_PERFORMANCE % python3 fork_join_lab1.py oversubscription
P= 1 | run1=0.000090s run2=0.000064s run3=0.000051s | avg=0.000068s
P= 2 | run1=0.000093s run2=0.000075s run3=0.000066s | avg=0.000078s
P= 4 | run1=0.000117s run2=0.000133s run3=0.000110s | avg=0.000120s
P= 8 | run1=0.000196s run2=0.000221s run3=0.000179s | avg=0.000199s
P= 16 | run1=0.000287s run2=0.000281s run3=0.000274s | avg=0.000280s
P= 32 | run1=0.000487s run2=0.000448s run3=0.000448s | avg=0.000461s
P= 64 | run1=0.000787s run2=0.000792s run3=0.000818s | avg=0.000799s
Saved timing table to task1_2_oversubscription.csv
(base) dimasaltynbek@MacBook-Pro-Dimas div2_lab_HIGH_PERFORMANCE %
```

task 1.3

```
((base) dimasaltynbek@MacBook-Pro-Dimas div2_lab_HIGH_PERFORMANCE % python3 fork_join_lab1.py saturation
Starting CPU saturation test with 4 threads.
While this runs, open Activity Monitor and watch CPU usage per core.
(This will take a while – CPU-bound work on each thread.)

Completed 4 parallel CPU-heavy tasks in 1.6804 seconds.
NOTE: Because of Python's GIL, threading does NOT give true CPU
parallelism for pure-Python CPU-bound code. Compare this time to
running cpu_heavy_work() 4 times sequentially (see below) to see
the GIL's effect firsthand.
Sequential baseline (4x, single thread): 1.7016 seconds
(base) dimasaltynbek@MacBook-Pro-Dimas div2_lab_HIGH_PERFORMANCE %
```

Q1.1 (Scheduling): Thread IDs print in a fixed but non-sequential order (0, 3, 1, 2) because of the artificial time.sleep() delay in the code, which depends on thread_id % 3. Threads with a 0ms delay (rank 0 and 3) finish first, then 1ms delay (rank 1), then 2ms delay (rank 2). Without this delay, the real order would depend on the OS thread scheduler, which decides which thread gets CPU time first — this is normally non-deterministic and can change between runs.

Q1.2 (Oversubscription): My data shows clear linear growth in overhead as P increases (0.068 ms at P=1 to 0.799 ms at P=64), even though my Mac only has 8 logical cores. When P > 8, the OS must time-share (context switch) between threads on the same core, which costs extra time for saving/restoring thread state and can cause cache thrashing (each context switch evicts the previous thread's data from the CPU cache).

Q1.3 (Barriers): The implicit barrier ensures all worker threads finish before the program continues — this guarantees the parallel region's results are complete and correct before they get used. Without it, the master thread could read partial or missing results, causing race conditions or crashes from using not-yet-computed data.

Q1.4 (Architectural Mapping): A hardware thread (SMT/Hyperthreading) is a real physical execution unit sharing a core's resources. An OS kernel thread is a software abstraction the OS scheduler manages and maps onto hardware threads. A language-level thread in Python (via threading) is built on top of OS kernel threads, but is limited by the GIL — my Task 1.3 result (1.68s parallel vs 1.70s sequential, basically no speedup) proves that Python threads don't give true CPU parallelism for pure-Python code, unlike real OS/hardware threads in C with OpenMP.

TASK 2:

```
((base) dimasaltynbek@MacBook-Pro-Dimas div2_lab_HIGH_PERFORMANCE % python3 pi_lab2.py race
Task 2.1: Race Condition Quantification (N=100,000,000)
Using numba(nogil=True) threads -> a REAL hardware race, unlike plain Python threading.

P=1 | Pi = 3.141592653590 | Error = 6.332712e-13
P=2 | Pi = 1.651730740518 | Error = 1.489862e+00
P=4 | Pi = 0.801191044448 | Error = 2.340402e+00
P=8 | Pi = 0.638264988265 | Error = 2.503328e+00

Saved task2_1_race_errors.csv
(base) dimasaltynbek@MacBook-Pro-Dimas div2_lab_HIGH_PERFORMANCE %
```

2.2:

```

(base) dimasaltynbek@MacBook-Pro-Dimas div2_lab_HIGH_PERFORMANCE % python3 pi_lab2.py critical
Task 2.2: Critical Section Overhead (N=1,000,000)
Serial (numba, 1 thread): Pi = 3.1415926536 | Time = 0.0010s
Critical section (4 threads): Pi = 3.1415926536 | Time = 0.1629s
Lock contention overhead: 16895.7% slower than serial baseline
Saved task2_2_critical_section.csv
(base) dimasaltynbek@MacBook-Pro-Dimas div2_lab_HIGH_PERFORMANCE %

```

TASK 2.3 + 2.4:

```

(base) dimasaltynbek@MacBook-Pro-Dimas div2_lab_HIGH_PERFORMANCE % python3 pi_lab2.py reduction
Task 2.3 + 2.4: Strong Scaling Benchmark (N=100,000,000), P in {1,2,4,8,16}
P= 1 | avg_time=0.1355s | S(P)=1.000 | E(P)=1.000 | Pi=3.14159265
P= 2 | avg_time=0.0605s | S(P)=2.240 | E(P)=1.120 | Pi=3.14159265
P= 4 | avg_time=0.0283s | S(P)=4.782 | E(P)=1.195 | Pi=3.14159265
P= 8 | avg_time=0.0256s | S(P)=5.291 | E(P)=0.661 | Pi=3.14159265
P=16 | avg_time=0.0227s | S(P)=5.958 | E(P)=0.372 | Pi=3.14159265

Saved task2_3_reduction_scaling.csv
(base) dimasaltynbek@MacBook-Pro-Dimas div2_lab_HIGH_PERFORMANCE %

```

Q2.1 (Memory Incoherency): A non-atomic += is really three CPU steps: LOAD (read value from memory), ADD (add the new term), STORE (write result back). If two threads interleave these steps — for example, both LOAD the same old value before either one STOREs — one thread's update gets overwritten and lost. This is exactly what happened in Task 2.1, where more threads meant more chances for this bad interleaving, and more lost updates.

Q2.2 (Reduction Trees): A critical section forces every single addition through one lock, one at a time — this is $O(P)$ because P threads all compete for the same lock, and only one at a time can proceed. A reduction tree lets each thread keep its own private partial sum with zero locking, then combines just P partial results at the end using a tree pattern (combine pairs, then combine pairs of pairs, etc.) — that only takes $O(\log P)$ combination steps. My data proves this: critical section was ~170x slower than serial, while reduction (Task 2.3) was actually faster than serial.

Q2.3 (Amdahl's Law): With 5% strictly serial ($p=0.95$), the theoretical max speedup at infinite processors is $S(\infty) = 1/(1-p) = 1/0.05 = 20x$. My measured speedup flattened much earlier — around 5.3–6.0x at $P=8$. Secondary factors: heterogeneous P/E cores on M1 (slower E-cores drag down the average), and memory bandwidth limits (all 8 cores competing for the same memory bus).

Q2.4 (Hardware Primitives): Modern CPUs implement atomic updates using instructions like LOCK ADD or CMPXCHG(compare-and-swap), which use the MESI cache protocol to guarantee only one core can modify a cache line at a time — the core "locks" the bus or cache line for the duration of the atomic operation, so no other core can interleave a conflicting read/write.

TASK 3:

```

(base) dimasaltynbek@MacBook-Pro-Dimas div2_lab_HIGH_PERFORMANCE % python3 mandelbrot_lab3.py static
Static scheduling baseline (numba prange, default row chunking)
Static render: 0.3476 seconds
(base) dimasaltynbek@MacBook-Pro-Dimas div2_lab_HIGH_PERFORMANCE %

```

task 3.1:

```
Static render: 0.1576 seconds
(base) dimasaltynbek@MacBook-Pro-Dimas div2_lab_HIGH_PERFORMANCE % python3 mandelbrot_lab3.py dynamic 4 64
Dynamic scheduling: threads=4, chunk=64
Dynamic render (threads=4, chunk=64): 0.2271 seconds
(base) dimasaltynbek@MacBook-Pro-Dimas div2_lab_HIGH_PERFORMANCE %
```

task 3.2:

```
(base) dimasaltynbek@MacBook-Pro-Dimas div2_lab_HIGH_PERFORMANCE % python3 mandelbrot_lab3.py sweep
Task 3.2: Parameter Sweep Matrix - P in {2,4,8,16} x Chunk in {1,16,64,256}
3 runs per cell, this will take a while...

P= 2 chunk= 1 | run1=0.4270 run2=0.6112 run3=0.4247 | avg=0.4876s
P= 2 chunk= 16 | run1=0.4131 run2=0.4100 run3=0.4141 | avg=0.4124s
P= 2 chunk= 64 | run1=0.4257 run2=0.4283 run3=0.4242 | avg=0.4260s
P= 2 chunk=256 | run1=0.4935 run2=0.4933 run3=0.4916 | avg=0.4928s
P= 4 chunk= 1 | run1=0.2134 run2=0.2378 run3=0.2791 | avg=0.2434s
P= 4 chunk= 16 | run1=0.2835 run2=0.2424 run3=0.2559 | avg=0.2606s
P= 4 chunk= 64 | run1=0.2319 run2=0.2941 run3=0.2553 | avg=0.2604s
P= 4 chunk=256 | run1=0.4926 run2=0.4953 run3=0.4909 | avg=0.4930s
P= 8 chunk= 1 | run1=0.1388 run2=0.1377 run3=0.1370 | avg=0.1378s
P= 8 chunk= 16 | run1=0.1366 run2=0.1379 run3=0.1411 | avg=0.1386s
P= 8 chunk= 64 | run1=0.2257 run2=0.2201 run3=0.2223 | avg=0.2227s
P= 8 chunk=256 | run1=0.4918 run2=0.4899 run3=0.4897 | avg=0.4905s
P=16 chunk= 1 | run1=0.1333 run2=0.1346 run3=0.1347 | avg=0.1342s
P=16 chunk= 16 | run1=0.1401 run2=0.1383 run3=0.1407 | avg=0.1397s
P=16 chunk= 64 | run1=0.2171 run2=0.2183 run3=0.2252 | avg=0.2202s
P=16 chunk=256 | run1=0.4900 run2=0.4882 run3=0.4956 | avg=0.4913s

Saved task3_2_sweep_matrix.csv
(base) dimasaltynbek@MacBook-Pro-Dimas div2_lab_HIGH_PERFORMANCE %
```

task 3.4:

```
(base) dimasaltynbek@MacBook-Pro-Dimas div2_lab_HIGH_PERFORMANCE % python3 mandelbrot_lab3.py imbalance 8 256
python3 mandelbrot_lab3.py imbalance 8 1
Task 3.4: Load Imbalance Metric (threads=8, chunk=256)
Per-thread total iterations: [np.int64(78238988), np.int64(122585384), np.int64(1313697), np.int64(140854), np.int64(0), np.int64(0), np.int64(0)]
Max=122585384 Min=0 Avg=25389609.9
Imbalance = (Max-Min)/Avg = 4.8282
Saved task3_4_imbalance.csv
Task 3.4: Load Imbalance Metric (threads=8, chunk=1)
Per-thread total iterations: [np.int64(28178649), np.int64(21419766), np.int64(25468708), np.int64(23385427), np.int64(25798882), np.int64(25684192), np.int64(24707110), np.int64(28474145)]
Max=28474145 Min=21419766 Avg=25389609.9
Imbalance = (Max-Min)/Avg = 0.2778
Saved task3_4_imbalance.csv
(base) dimasaltynbek@MacBook-Pro-Dimas div2_lab_HIGH_PERFORMANCE %
```

Q3.1 (Fine-Grained Contention): In theory, chunk=1 should be slow because every thread must lock the shared counter after every single row. But in my results, chunk=1 was actually the FASTEST option. This is because each "unit of work" here is one full image row (1920 pixels, up to 1000 iterations each) — this is a large amount of real computation compared to the tiny cost of locking a counter. The lock is only contested for a few CPU instructions, so it doesn't create a real bottleneck here. Fine-grained contention would only show up if each work unit was much smaller (e.g., a single pixel).

Q3.2 (Spatial Imbalance): In static scheduling, threads assigned to the middle rows of the image become stragglers, because the Mandelbrot set (the black region) sits in the center of the complex plane. Points inside the set need the full 1000 iterations before the loop stops, while points outside escape after just a few. Static scheduling with default chunking gives each thread a fixed, equal-sized block of consecutive rows — the threads whose rows fall mostly in the middle (near the Mandelbrot set) get much more work than threads handling the edges of the image.

Q3.3 (Guided Policy): `schedule(guided, chunk)` starts by giving out large chunks (fast, low locking overhead) when there's plenty of work left, then automatically shrinks the chunk size (down to the minimum chunk value) as the work queue empties. This way, most of the work is distributed cheaply in big pieces early on, and only near the end — when load imbalance matters most — does it switch to small chunks for fine-grained balancing. It combines static's low overhead early with dynamic's balancing at the end.

Q3.4 (Engineering Decision Framework): Use **static** when all iterations cost about the same (uniform workload) — it has near-zero scheduling overhead. Use **dynamic** when the workload is unpredictable or highly non-uniform (like Mandelbrot) — my data shows chunk=1 gave almost perfect balance (0.28) vs chunk=256's terrible balance (4.83). Use **guided** as a safe middle ground when you're not sure how uniform the workload is, or want low overhead without manually tuning chunk size.

TASK 4:

task 4.1-4.2:

```
(base) dimasaltynbek@MacBook-Pro-Dimas div2_lab_HIGH_PERFORMANCE % python3 false_sharing_lab4.py scaling
Task 4.1 + 4.2: Unpadded vs Padded scaling (cache line = 128 bytes, stride = 16)
ITERATIONS per thread = 100,000,000

P= 1 | unpadded=0.0439s | padded=0.0440s | ratio=0.999x
P= 2 | unpadded=0.0455s | padded=0.0455s | ratio=1.000x
P= 4 | unpadded=0.0475s | padded=0.0472s | ratio=1.007x
P= 8 | unpadded=0.0709s | padded=0.0710s | ratio=1.000x
P=16 | unpadded=0.1473s | padded=0.1446s | ratio=1.018x

Saved task4_1_2_scaling.csv
(base) dimasaltynbek@MacBook-Pro-Dimas div2_lab_HIGH_PERFORMANCE %
```

task 4.3:

```
Saved task4_1_2_scaling.csv
(base) dimasaltynbek@MacBook-Pro-Dimas div2_lab_HIGH_PERFORMANCE % python3 false_sharing_lab4.py threadl
Task 4.3: Thread-Local Accumulator variant (register-based, single write at end)
P= 1 | thread_local=0.0442s
P= 2 | thread_local=0.0458s
P= 4 | thread_local=0.0475s
P= 8 | thread_local=0.0850s
P=16 | thread_local=0.1474s

Saved task4_3_threadlocal.csv
Compare this file with task4_1_2_scaling.csv - thread-local should be the fastest of all three variants.
(base) dimasaltynbek@MacBook-Pro-Dimas div2_lab_HIGH_PERFORMANCE %
```

Q4.1 (MESI State Transitions): Imagine a 128-byte cache line (my machine's actual size) holding several `int64` counters close together. When Core 0 writes to its counter, the MESI protocol moves that cache line into the "Modified" state on Core 0, and any copy on Core 1 becomes "Invalid". If Core 1 then writes to its own (different) counter on the same line, it must first fetch the line from Core 0 (a coherence transaction), moving it back to "Modified" on Core 1 and invalidating Core 0's copy. This ping-pong repeats every time either thread writes, even though the two counters are logically unrelated.

Q4.2 (Bus Bouncing): In a typical false-sharing scenario, adding more cores makes things slower because each core's write forces every other core sharing that cache line to reload it — the "physical bottleneck" is the interconnect/bus bandwidth and latency used for cache-line coherence traffic, not the actual arithmetic. Interestingly, on my Apple M1, my measured results did NOT show this slowdown (ratio ≈ 1.0 between padded and unpadded) — this suggests M1's cache-coherency fabric is fast and wide enough that this bottleneck barely appears for this workload size.

Q4.3 (True vs. False Sharing): True sharing is when two threads access the *same* variable — this is a real logical dependency, and any synchronization needed (locks, atomics) is necessary for correctness. False sharing is when two threads access *different, unrelated*

variables that just happen to physically sit on the same cache line — there's no logical dependency at all, but the hardware still treats it like a conflict because it only tracks coherence at the granularity of a whole cache line, not individual variables.

Q4.4 (Adjacent Line Prefetching): The JVM's @Contended annotation adds 128 bytes of padding by default (not 64) because many modern x86-64 CPUs use **adjacent line prefetching** — when a core loads one 64-byte cache line, the hardware prefetcher automatically also loads the *next* 64-byte line, effectively making the "false sharing danger zone" 128 bytes wide instead of 64. Padding by 128 bytes guarantees no adjacent-line prefetch can pull in another thread's data by accident. This matches my own machine, where `sysctl hw.cachelinesize` directly reports 128 bytes as the real line size, so I didn't even need the "double it for prefetching" rule of thumb — 128 was already the true value.